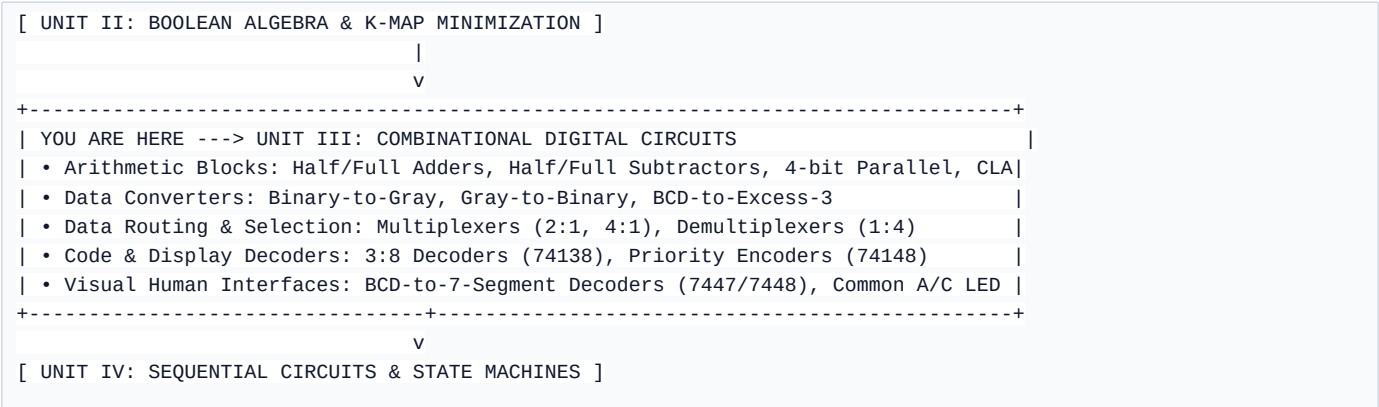


DEC Unit III: Combinational Circuits

Comprehensive First-Principles Lecture Notes

Unit III: Combinational Digital Circuits

Master Subject Map: Unit III Position



In 3 lines: In Unit II, we mastered logic minimization on paper. Unit III combines those minimized gates into practical computing blocks called **Combinational Circuits**. These memoryless circuits perform instantaneous calculations (Adders/Subtractors), route data streams (Multiplexers), and translate binary code into human-readable symbols (Seven-Segment Displays).

Prerequisites for this Unit

Before studying Unit III, ensure you recall these concepts: 1. **Universal & Exclusive Gates (Unit I, Sections 1.5–1.7):** XOR ($A \oplus B = AB + \overline{A}\overline{B}$), AND (AB), OR ($A+B$), and Inverters. 2. **K-Map Simplification (Unit II, Section 2.7):** Grouping cells to produce minimal Sum-of-Products equations. 3. **Binary Arithmetic & 2's Complement Subtraction (Unit I, Sections 1.2–1.3):** Converting subtraction into controlled addition. 4. **Gray Code & BCD Rules (Unit I, Section 1.4):** Single-bit transitions and 4-bit decimal representations.

3.1 Combinational Logic: Definition and Formal Design Procedure

Source: Kumar (Ch. 7, pp. 357–358); Mano (Ch. 4, pp. 164–172)
[Pacing: Methodical Framework — Learn the Standard Design Steps]

Need (Problem-First)

How do professional engineers transform a fuzzy English problem statement (e.g., "design a voting machine for a 3-member committee where the majority wins") into a functional electronic schematic? We need a universal, repeatable step-by-step design procedure.

Physical Definition of Combinational Logic

A **Combinational Circuit** is a connected network of logic gates whose output voltages at any given instant of time depend **strictly and solely on the input voltages applied at that exact same instant** (Kumar, p. 357).
- It has **no feedback loops** (outputs are never fed back to inputs). - It has **no memory elements** (it cannot remember what happened one microsecond ago). - It is entirely governed by combinational Boolean functions: $Y_j = f_j(I_1, I_2, \text{dots},$

I_n).

The Standard 5-Step Design Procedure (Kumar, p. 358; Mano, p. 170):

- 1. **Problem Formulation:** State the circuit's behavior clearly in words, identifying all required input and output variables.
- 2. **Variable Assignment:** Assign algebraic letter symbols (e.g., A, B, C for inputs; X, Y, S for outputs).
- 3. **Truth Table Construction:** Exhaustively list all 2^n input combinations and define the required output for each row.
- 4. **Logic Minimization:** Plot each output variable onto a Karnaugh Map and derive the minimal Boolean expression (in SOP or POS).
- 5. **Logic Schematic Realization:** Draw the circuit diagram using standard gate symbols, or convert to universal NAND/NOR logic for fabrication.

PART 1: Arithmetic Building Blocks (Adders, Subtractors, and Lookahead Carry)

3.2 The Half Adder and Full Adder

Source: Kumar (Ch. 7, pp. 358–363); Mano (Ch. 4, pp. 173–178); Floyd (Ch. 5, pp. 248–256)

[Pacing: Fundamental Arithmetic — Study Every Derivation Step]

Need (Problem-First)

All computing—from calculating 3D graphics in video games to processing banking transactions—boils down to basic binary addition. Without a circuit that can add two electrical bits and handle carries, modern computers cannot exist.

1. The Half Adder (Kumar, pp. 358–360)

- **Definition:** A combinational circuit that adds two single binary bits (A and B), producing two outputs: a Sum bit (S) and a Carry bit (C). It is called a "half" adder because it has no provision to accept a carry arriving from a previous lower stage.
- **Truth Table (Kumar, p. 358):**

Input A	Input B	Sum (S)	Carry (C)	Arithmetic Meaning
0	0	0	0	0 + 0 = 0
0	1	1	0	0 + 1 = 1
1	0	1	0	1 + 0 = 1
1	1	0	1	1 + 1 = 10_2 (Sum 0, Carry 1)

- **Boolean Equations:**
- Notice the Sum column: S is 1 only when inputs are different ■ XOR Gate:

S = AB + AB = A ⊕ B

- Notice the Carry column: C is 1 only when both inputs are 1 ■ AND Gate:

C = AB

- **Authentic Diagram (Kumar, p. 359, Figure 7.3):**

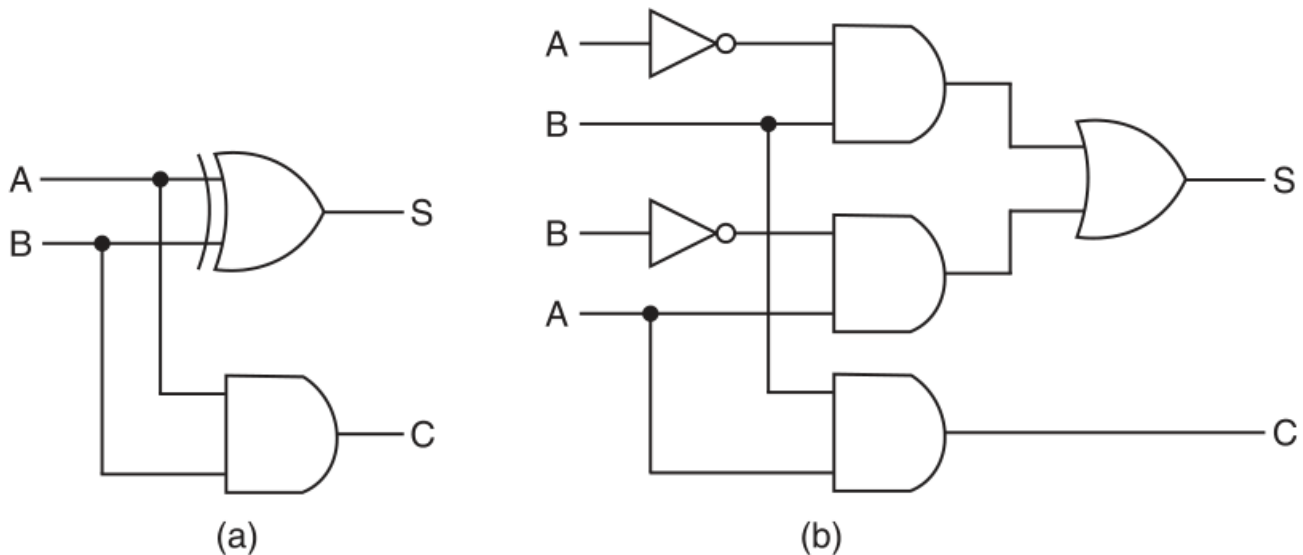


Figure 7.3 Logic diagrams of half-adder.

Figure

7.3: Half-adder circuit implementations: (a) using basic AND, OR, NOT gates; (b) using an XOR gate and an AND gate. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 359 (printed p. 327).

2. The Full Adder (Kumar, pp. 360–363)

- **Need:** When adding multi-bit binary numbers (e.g., **1101 + 1011**), each column after the LSB must add **three bits simultaneously**: the augend bit (**A**), the addend bit (**B**), and the **incoming carry (C_{in})** generated by the previous column! A half adder cannot do this. We need a **Full Adder**.
- **Truth Table (Kumar, p. 360):**

A	B	C_{in}	Sum (S)	Carry Out (C_{out})	Arithmetic Sum ($A + B + C_{in}$)
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	1	0	1
0	1	1	0	1	2 = 10 ₂
1	0	0	1	0	1
1	0	1	0	1	2 = 10 ₂
1	1	0	0	1	2 = 10 ₂
1	1	1	1	1	3 = 11 ₂

- **Derivation of Boolean Expressions:**
- **Sum (S):** S is 1 whenever an **odd number of inputs are 1** (parity function):

$$S = A \overline{B} \overline{C_{in}} + A \overline{B} C_{in} + A B \overline{C_{in}} + A B C_{in} = A \oplus B \oplus C_{in}$$

- **Carry Out (C_{out}):** C_{out} is 1 whenever **two or more inputs are 1**:

$$C_{\text{out}} = AB + BC_{\text{in}} + AC_{\text{in}} = AB + C_{\text{in}}(A \oplus B)$$

• **Realization Using Two Half Adders (Kumar, p. 361, Figure 7.7):**

A Full Adder can be constructed by cascading two Half Adders and one OR gate:

- Half Adder 1 adds **A** and **B**, generating partial sum **S₁ = A ⊕ B** and carry **C₁ = AB**.
- Half Adder 2 adds **S₁** and **C_{in}**, generating final sum **S = (A ⊕ B) ⊕ C_{in}** and carry **C₂ = (A ⊕ B)C_{in}**.
- The final carry out is formed by ORing the two partial carries: **C_{out} = C₁ + C₂ = AB + C_{in}(A ⊕ B)**.

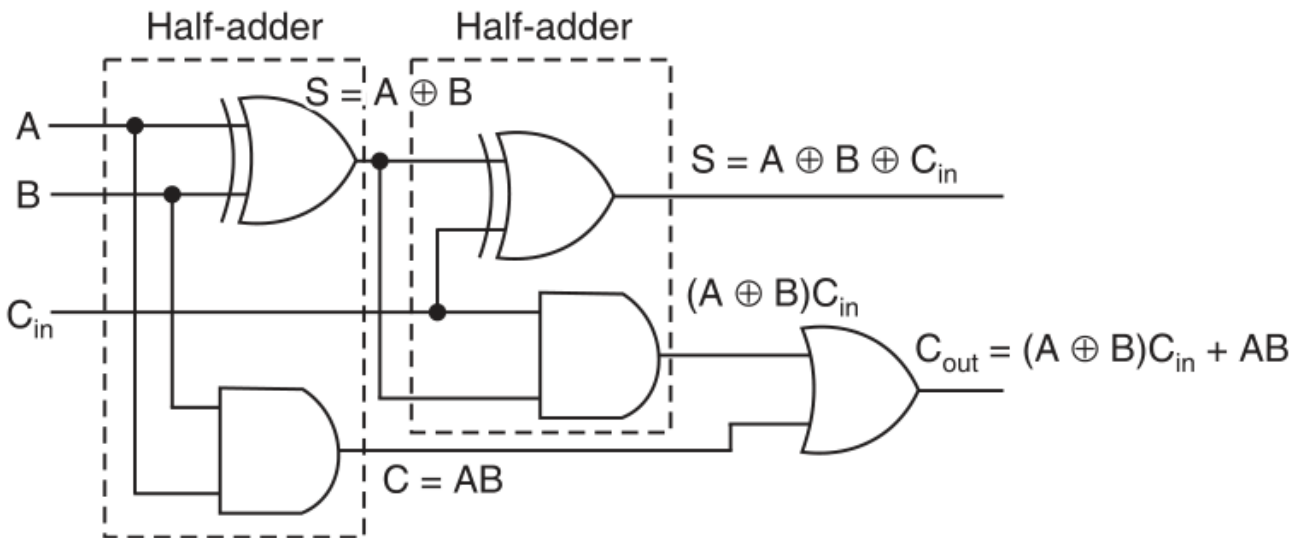


Figure 7.7 Logic diagram of a full-adder using two half-adders.

Figure

7.7: Construction of a Full Adder using two cascaded Half Adders and an OR gate. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 361 (printed p. 329).

3.3 The Half Subtractor and Full Subtractor

Source: Kumar (Ch. 7, pp. 363–367); Mano (Ch. 4, pp. 178–181)

[Pacing: Parallel Structure — Compare Direct Subtractors to Adders]

1. The Half Subtractor (Kumar, pp. 363–365)

- **Definition:** A circuit that subtracts bit **B** from bit **A**, producing a **Difference (D)** and a **Borrow out (B_{out})**.
- **Truth Table & Equations:**
- **D = AB + AB̄ = A ⊕ B** (Notice: Difference equation is identical to Half Adder Sum!)
- **B_{out} = AB** (Borrow is needed only when subtracting 1 from 0: 0 - 1 = 1 with borrow 1).

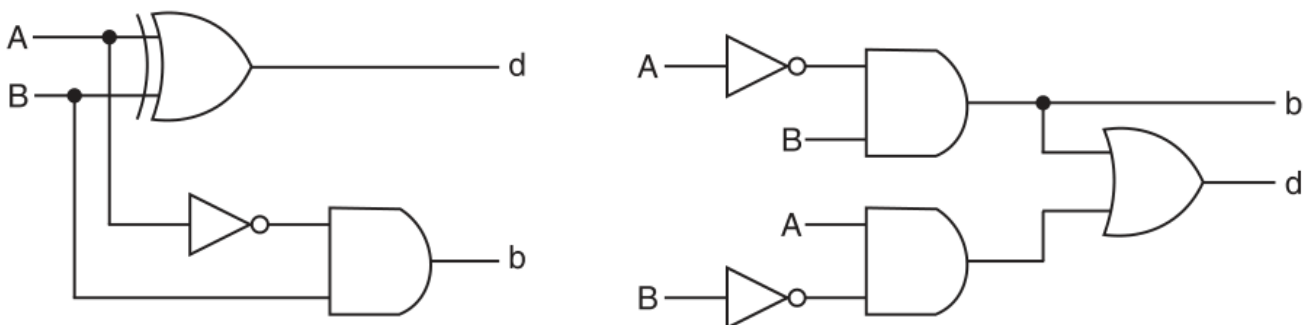


Figure 7.13 Logic diagrams of a half-subtractor.

Figure

7.13: Logic diagrams of a Half Subtractor using basic gates and XOR-AND configuration. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 364 (printed p. 332).

2. The Full Subtractor (*Kumar*, pp. 365–367)

- **Definition:** Subtracts bits **B** and incoming borrow **B_{in}** from bit **A** ($A - B - B_{in}$).
- **Boolean Equations:**
- $D = A \oplus B \oplus B_{in}$
- $B_{out} = AB + AB_{in} + BB_{in} = AB + B_{in}(A \oplus B)$

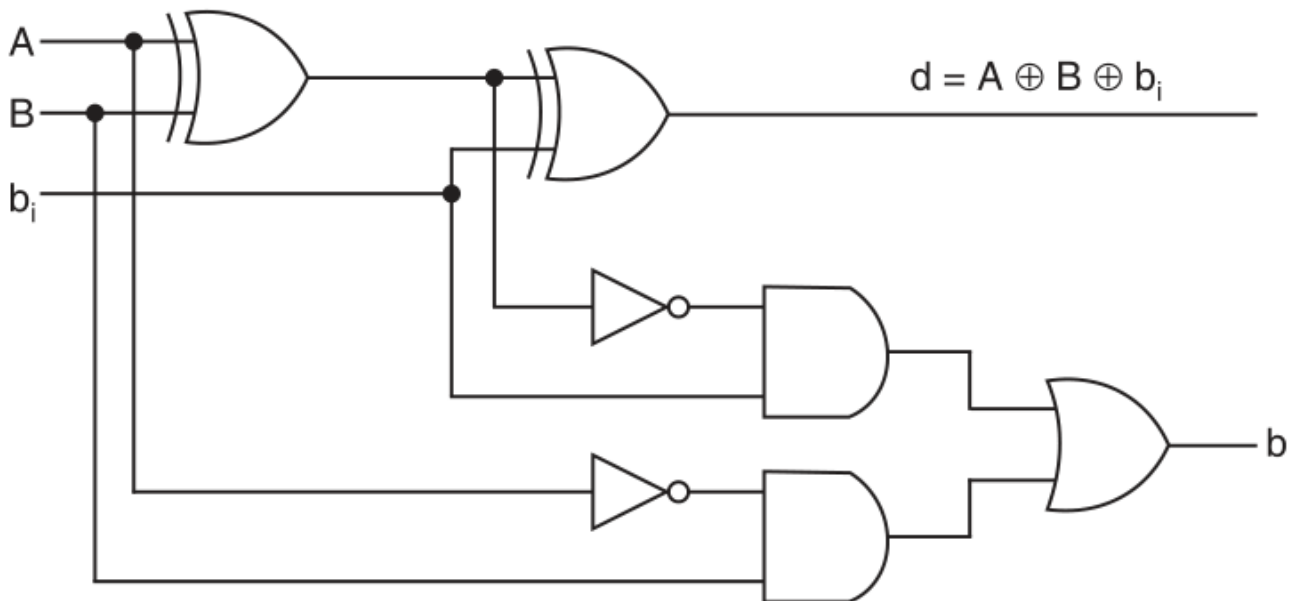


Figure 7.17 Logic diagram of a full-subtractor.

Figure

7.17: Logic diagram of a Full Subtractor constructed using two Half Subtractors and an OR gate. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 366 (printed p. 334).

3.4 Multi-Bit Adders and the 4-Bit Parallel Adder/Subtractor

Source: *Kumar* (Ch. 7, pp. 367–373); *Mano* (Ch. 4, pp. 173–178); *Floyd* (Ch. 5, pp. 250–256)

[Pacing: Hardware Realization — High Practical Value]

1. The 4-Bit Binary Parallel Adder (Ripple Carry Adder) (*Kumar*, p. 367)

To add two 4-bit binary numbers $A = A_3 A_2 A_1 A_0$ and $B = B_3 B_2 B_1 B_0$, four Full Adders (FA) are chained together in parallel (*Kumar*, Figure 7.20):

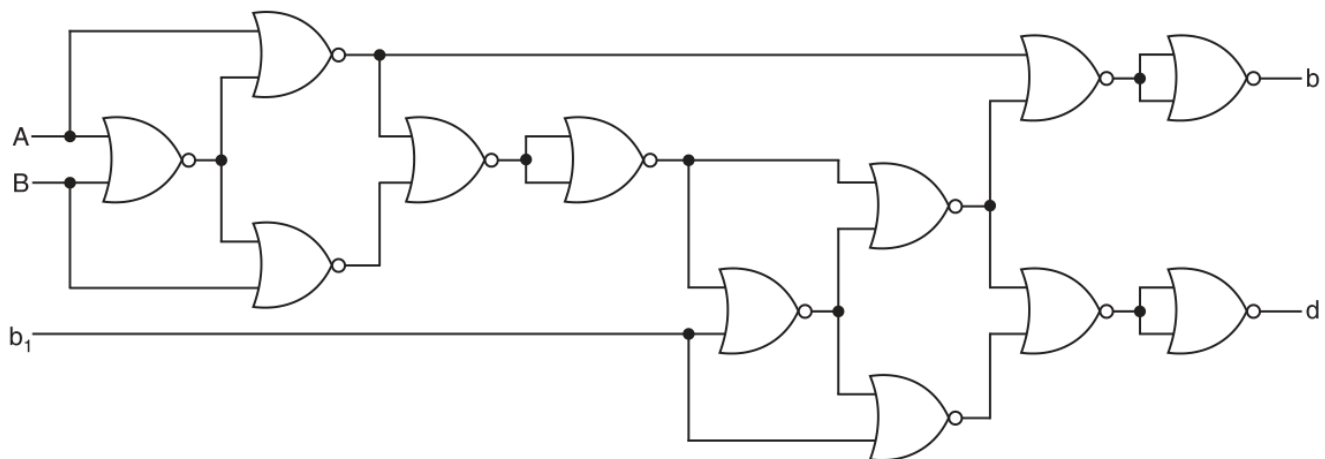


Figure 7.19 Logic diagram of a full subtractor using only 2-input NOR gates.

7.5 BINARY PARALLEL ADDER

A binary parallel adder is a digital circuit that adds two binary numbers in parallel form and produces the arithmetic sum of those numbers in parallel form. It consists of full adders connected in a chain, with the output carry from each full-adder connected to the input carry of the next full-adder in the chain.

Figure 7.20 shows the interconnection of four full-adder (FA) circuits to provide a 4-bit parallel adder. The augend bits of A and addend bits of B are designated by subscript numbers from right to left, with subscript 1 denoting the lower-order bit. The carries are connected in a chain through the full-adders. The input carry to the adder is C_{in} and the output carry is C_4 . The S outputs generate the required sum bits. When the 4-bit full-adder circuit is enclosed within an IC package, it has four terminals for the augend bits, four terminals for the addend bits, four terminals

Figure

7.20: Four-bit parallel binary adder (Ripple Carry Adder) chaining four Full Adders together. Sourced from A. Anand Kumar, Fundamentals of Digital

Circuits (4th Ed.), p. 367 (printed p. 335).

- Stage 0 adds **A₀**, **B₀** and input carry **C₀**, producing Sum **S₀** and carry **C₁**.
- Carry **C₁** ripples into Stage 1; carry **C₂** ripples into Stage 2; carry **C₃** ripples into Stage 3.
- Standard IC: **7483 / 74LS283** (4-bit binary full adder).

2. The Combined 4-Bit Parallel Adder/Subtractor (Kumar, p. 369, Figure 7.22)

Why build a separate adder and a separate subtractor chip when you can make a single circuit perform both?

Recall from Unit I that:

$$A - B = A + (2\text{'s complement of } B) = A + B + 1$$

We can achieve this mathematically and physically with a single control line labeled **M (Mode / ADD/overLine{SUB})** and four **XOR** gates:

Figure 7.22 shows a 4-bit adder-subtractor circuit. Here the addition and subtraction operations are combined into one circuit with one common binary adder. This is done by including an X-OR gate with each full-adder. The mode input M controls the operation. When $M = 0$, the circuit is an adder, and when $M = 1$, the circuit becomes a subtractor. Each X-OR gate receives input M and one of the inputs of B . When $M = 0$, we have $B \oplus 0 = B$. The full-adder receives the value of B , the input carry is 0 and the circuit performs $A + B$. When $M = 1$, we have $B \oplus 1 = \bar{B}$ and $C_1 = 1$. The B inputs are complemented and a 1 is added through the input carry. The circuit performs the operation A plus the 2's complement of B .

Figure

7.22: Complete 4-bit binary adder-subtractor utilizing XOR gates as programmable inverters and 2's complement carry injection. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 369 (printed p. 337).

Characteristic 13 Walkthrough: Hardware Reality Check

- **When Mode $M = 0$ (Addition Mode):**
 - Each XOR gate receives B_i and 0 . Because $B_i \oplus 0 = B_i$, the B inputs pass through completely **uninverted**.
 - The initial carry input C_0 receives $M = 0$.
 - The circuit computes: **Output = $A + B + 0 = A + B$. It performs pure binary addition!**
- **When Mode $M = 1$ (Subtraction Mode):**
 - Each XOR gate receives B_i and 1 . Because $B_i \oplus 1 = \bar{B}_i$, the XOR gates act as inverters, producing the **1's complement: $\bar{B}$** !
 - Simultaneously, the initial carry input C_0 receives $M = 1$, automatically injecting the required **+1** into the LSB!
 - The circuit computes: **Output = $A + B + 1 = A + (-B) = A - B$. It performs pure 2's complement subtraction!**
 - A single wire flips the entire calculating machine between addition and subtraction!

3.5 High-Speed Arithmetic: The Carry Lookahead Adder (CLA)

Source: Kumar (Ch. 7, pp. 369–373); Mano (Ch. 4, pp. 176–178)

[Pacing: Conceptually Heavy — The Fundamental Speed Bottleneck in Computing]

Need (Problem-First: The Ripple Delay Bottleneck)

In the 4-bit ripple adder (Figure 7.20), Stage 3 **cannot calculate its sum bit S_3 until Stage 2 finishes, which cannot finish until Stage 1 finishes, which waits on Stage 0!** - If each full adder has a gate propagation delay of $t_{\{pd\}} = 5 \text{ ns}$, a 4-bit adder takes $4 \times 5 = 20 \text{ ns}$. - But in a modern 64-bit microprocessor ALU, a 64-bit ripple adder would take $64 \times 5 \text{ ns} = 320 \text{ ns}$! The processor could not exceed a dismal clock speed of **3 MHz!** - We need a circuit where **all carries are calculated simultaneously in parallel** without waiting for previous stages to ripple. This is the **Carry Lookahead Adder (CLA)**.

The Mathematical Formulation: Generate and Propagate

For any adder stage i , define two independent functions (Kumar, p. 370): 1. **Carry Generate (G_i):**

$$G_i = A_i \cdot B_i$$

Physical Meaning: If both A_i and B_i are **1**, this stage **generates a carry internally**, regardless of whether an incoming carry arrived from earlier stages. 2. **Carry Propagate (P_i):**

$$P_i = A_i \oplus B_i$$

Physical Meaning: If either A_i or B_i is **1**, an incoming carry C_i will be **propagated straight through** to become an outgoing carry $C_{\{i+1\}}$.

The carry output of any stage can thus be written as:

$$C_{\{i+1\}} = G_i + P_i C_i$$

Unrolling the Carries Directly:

Now, expand each carry stage by algebraic substitution without waiting for ripples: - $C_1 = G_0 + P_0 C_0$ - $C_2 = G_1 + P_1 C_1 = \mathbf{G_1 + P_1 G_0 + P_1 P_0 C_0}$ - $C_3 = G_2 + P_2 C_2 = \mathbf{G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0}$ - $C_4 = G_3 + P_3 C_3 = \mathbf{G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0}$

Look at those equations! Notice that C_4 depends ONLY on the initial input carry C_0 and the immediate input bits A_i , B_i via P_i and G_i !

Every single carry bit (C_1 , C_2 , C_3 , C_4) is generated through a two-level AND-OR gate network simultaneously in parallel!

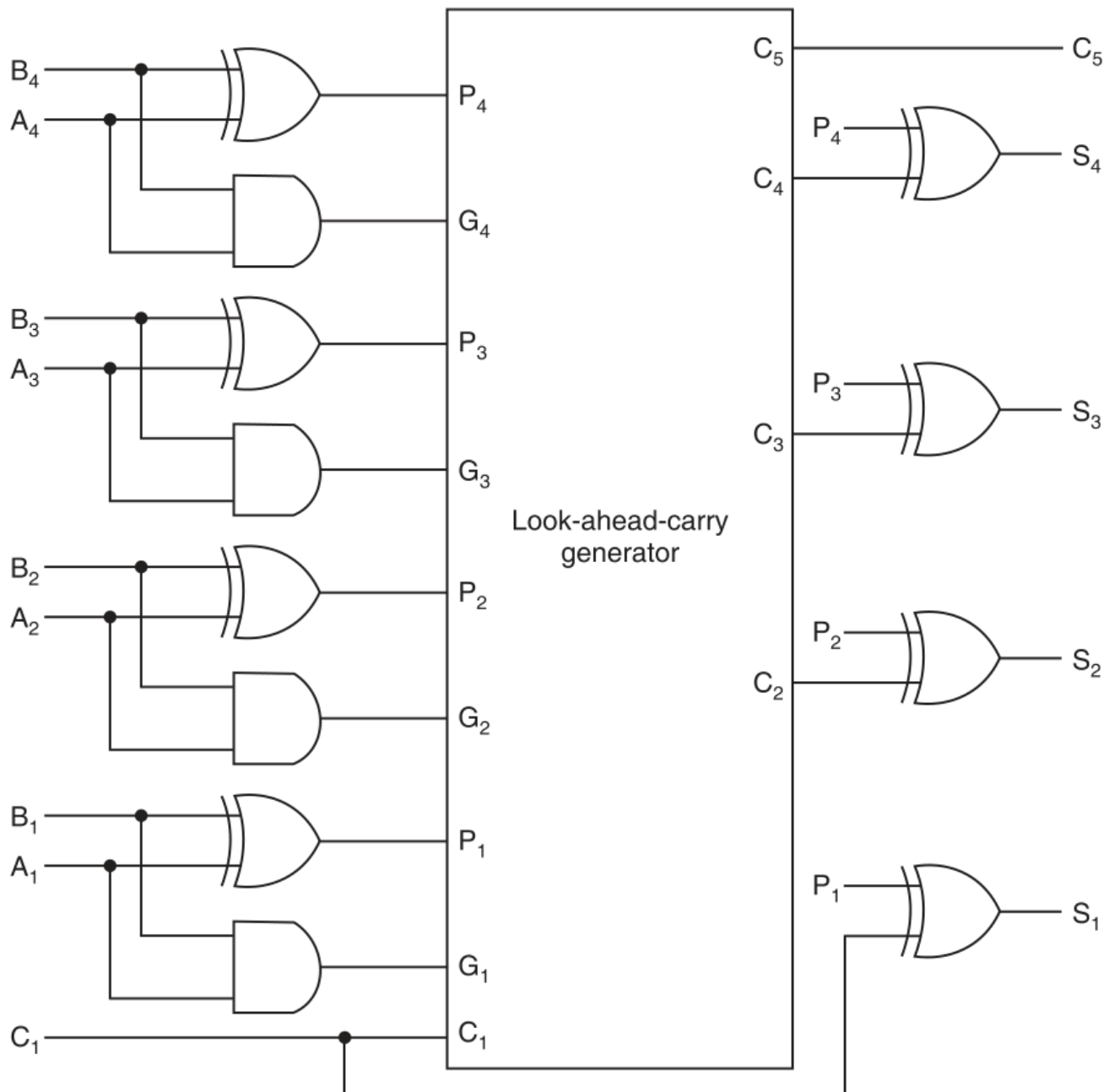


Figure 7.24 Logic diagram of a 4-bit look-ahead-carry adder.

Figure

7.24: Logic diagram of a 4-bit Carry Lookahead Adder (CLA) eliminating serial ripple delays. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 371 (printed p. 339).

- **Speed Comparison:** Regardless of whether the adder is 4-bit, 16-bit, or 64-bit, the lookahead carry generator calculates all carries in just **two gate delays** (≈ 1 to 2 ns), boosting computation speed by over **1000%**!

PART 2: Code Converters and Data Selectors (Multiplexers / Demultiplexers)

3.6 Hardware Code Converters: Binary, Gray, and Excess-3

Source: Kumar (Ch. 7, pp. 382–388); Mano (Ch. 4, pp. 171–173)

[Pacing: Systematic Design — Practice the 5-Step Method]

1. 4-Bit Binary-to-Gray Code Converter (Kumar, p. 383)

Following the mathematical rule derived in Unit I ($G_3 = B_3$; $G_2 = B_3 \oplus B_2$; $G_1 = B_2 \oplus B_1$; $G_0 = B_1 \oplus B_0$), the physical hardware requires exactly **three XOR gates**:

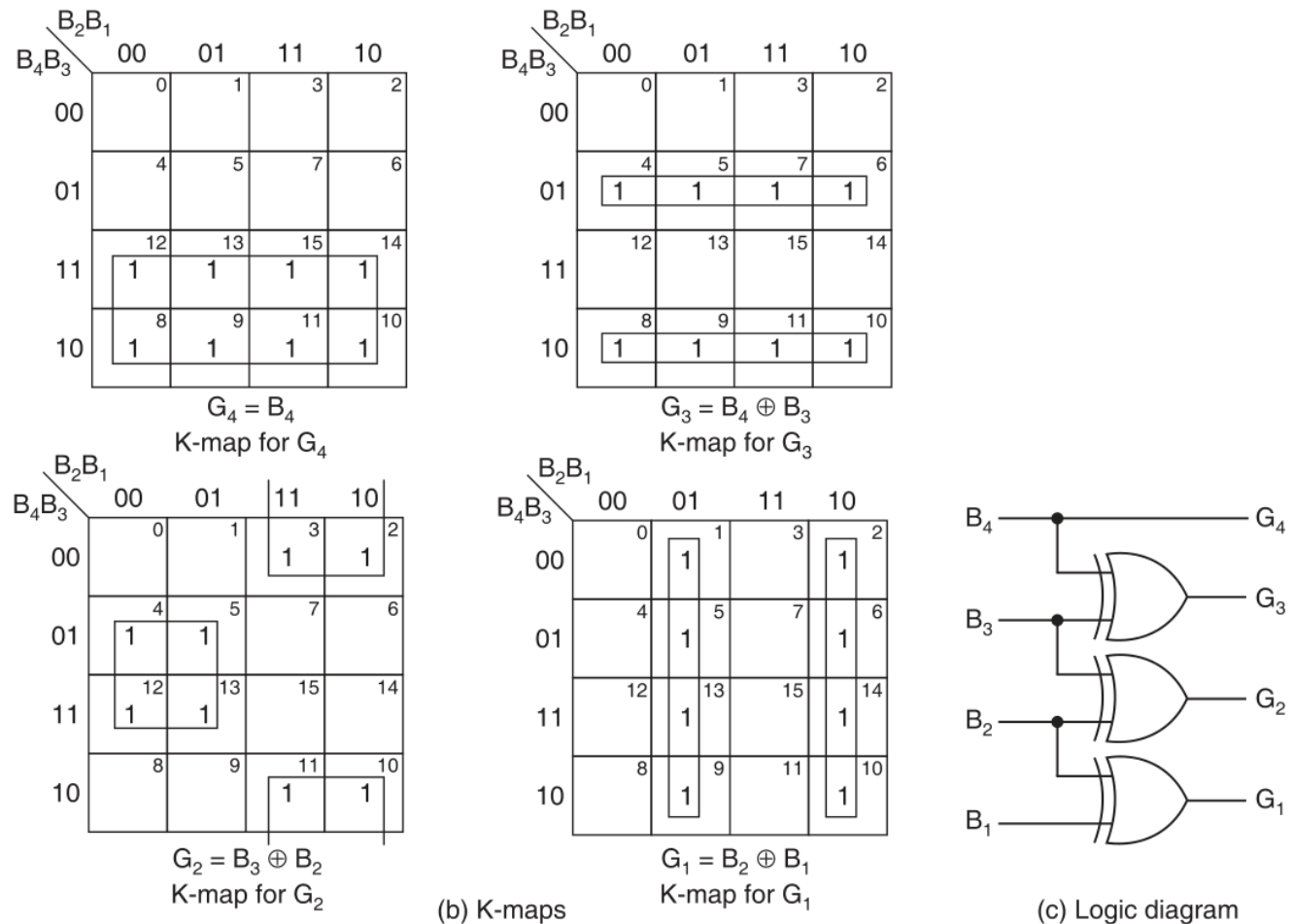


Figure 7.32 4-bit binary-to-Gray code converter.

Figure

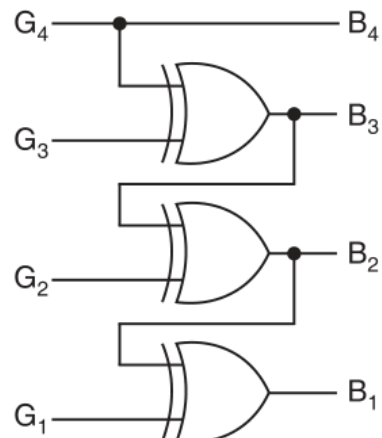
7.32: Logic diagram of a 4-bit Binary-to-Gray code converter using XOR gates. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 383 (printed p. 351).

2. 4-Bit Gray-to-Binary Code Converter (Kumar, p. 384)

Following the inverse rule ($B_3 = G_3$; $B_2 = B_3 \oplus G_2$; $B_1 = B_2 \oplus G_1$; $B_0 = B_1 \oplus G_0$), the circuit cascades three XOR gates, feeding each computed binary bit forward into the next lower stage:

4-bit Gray				4-bit binary			
G_4	G_3	G_2	G_1	B_4	B_3	B_2	B_1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
0	1	0	1	0	1	1	0
0	1	0	0	0	1	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	1	1	0	1	0
1	1	1	0	1	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	0	0	1	1	1	1	0
1	0	0	0	1	1	1	1

(a) Conversion table



(c) Logic diagram

G_2G_1					
		00	01	11	10
G_4G_3	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10

$$B_4 = G_4$$

K-map for B_4

G_2G_1					
		00	01	11	10
G_4G_3	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10

$$B_3 = G_4 \oplus G_3$$

K-map for B_3

G_2G_1					
		00	01	11	10
G_4G_3	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10

$$B_2 = G_4 \oplus G_3 \oplus G_2$$

K-map for B_2

G_2G_1					
		00	01	11	10
G_4G_3	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10

$$B_1 = G_4 \oplus G_3 \oplus G_2 \oplus G_1$$

K-map for B_1

(b) K-maps

Figure 7.33 4-bit Gray-to-binary code converter.

Figure 7.33: Logic

diagram of a 4-bit Gray-to-Binary code converter. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 384 (printed p. 352).

3. BCD-to-Excess-3 Code Converter (Kumar, p. 387)

By setting up a truth table adding **0011_2 (3)** to each valid BCD digit (**0000** to **1001**) and assigning don't cares (**X**) to illegal BCD rows **10** to **15**, four K-maps yield the optimized hardware equations:

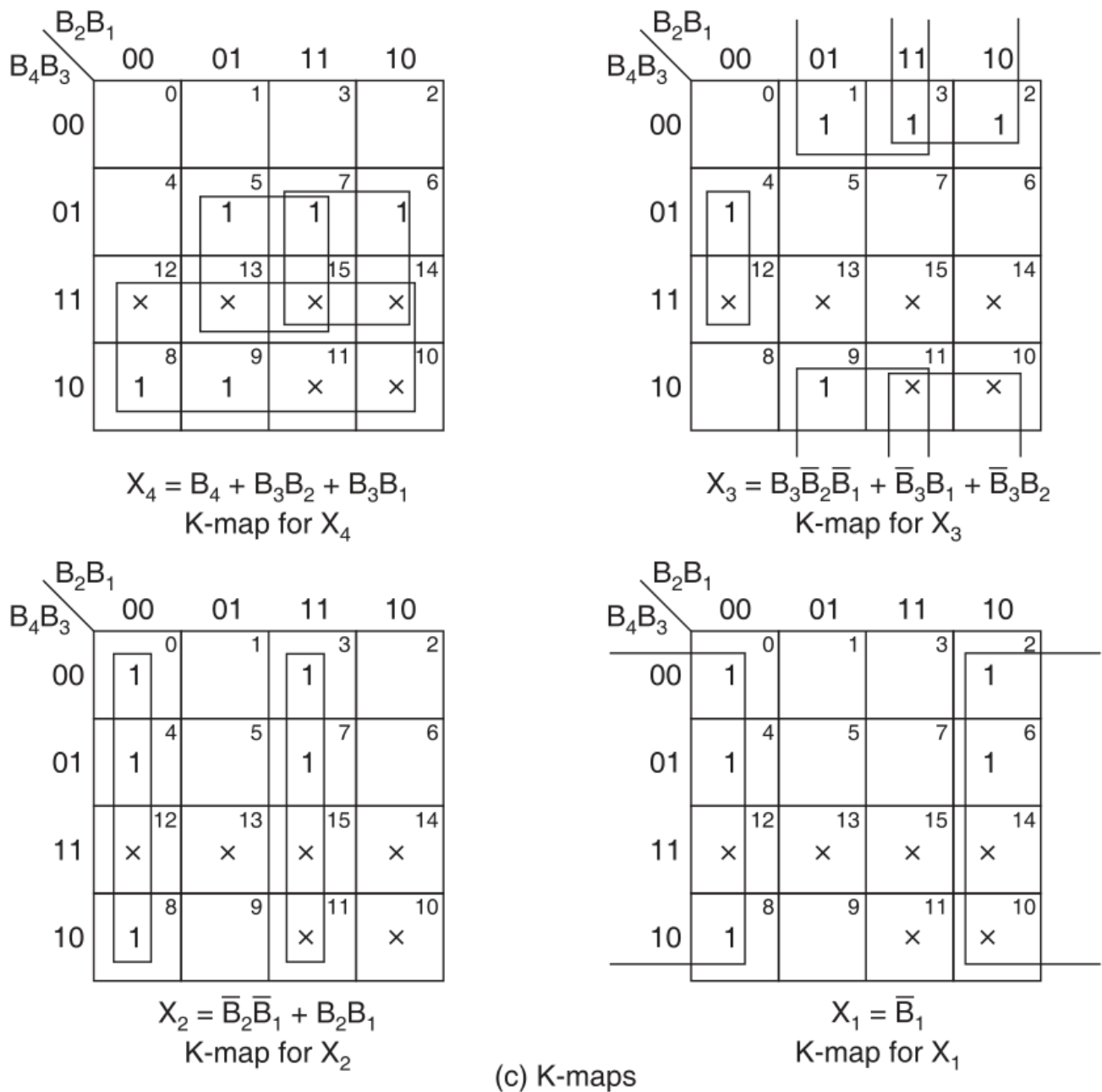


Figure 7.35 4-bit BCD-to-XS-3 code converter.

Figure

7.35: Logic diagram of a 4-bit BCD-to-Excess-3 code converter derived via K-map minimization. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 387 (printed p. 355).

3.7 Multiplexers (Data Selectors) and Demultiplexers

Source: Kumar (Ch. 7, pp. 421–434); Mano (Ch. 4, pp. 199–206); Floyd (Ch. 5, pp. 268–276)

[Pacing: Core Digital Routing — High Examination & Practical Value]

Need (Problem-First)

Imagine a digital telephone exchange where four different callers want to send audio data across a single transatlantic cable wire. Running four separate undersea cables costs millions of dollars. How can multiple data sources share a single communication line without their signals tangling? We need an electronic rotary switch: a **Multiplexer (MUX)** at the transmitting end, and a **Demultiplexer (DEMUX)** at the receiving end.

1. The Multiplexer (MUX / Data Selector) Defined

A **Multiplexer** is a combinational circuit that selects binary information from one of many input data lines and directs it to a single output line (Kumar, p. 421). - **The Rule of Inputs:** A multiplexer with 2^n data input lines requires n select control lines (S) to specify which input is routed to the output. - **2:1 MUX** ■ 2^1 data inputs, 1 select line. - **4:1 MUX** ■ 2^2 data inputs, 2 select lines (S₁, S₀). - **8:1 MUX** ■ 2^3 data inputs, 3 select lines (S₂, S₁, S₀).

The 2-Input Multiplexer (2:1 MUX) (Kumar, p. 422, Figure 7.76)

- Logic equation: $Y = S D_0 + S D_1$.
- When $S = 0$: $Y = (1)D_0 + (0)D_1 = D_0$. Output follows data line D₀.
- When $S = 1$: $Y = (0)D_0 + (1)D_1 = D_1$. Output follows data line D₁.

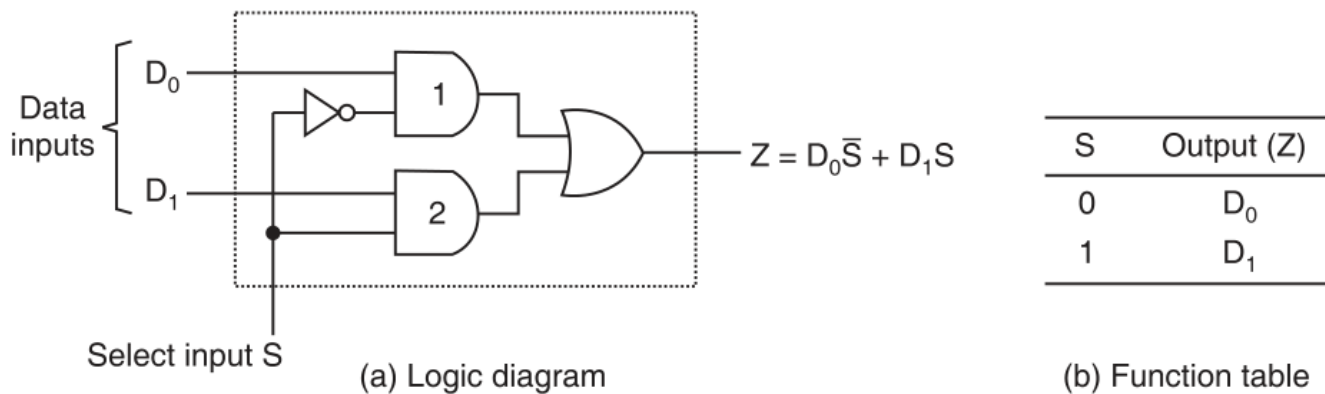


Figure 7.76 2-input multiplexer.

Figure

7.76: Logic circuitry and function table of a 2-input multiplexer. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 422 (printed p. 390).

The 4-Input Multiplexer (4:1 MUX) (Kumar, p. 422, Figure 7.77)

- Four data inputs (D₀, D₁, D₂, D₃), two select inputs (S₁, S₀), and one Enable input (E / Strobe):

$$Y = S_1 S_0 D_0 + S_1 S_0 D_1 + S_1 S_0 D_2 + S_1 S_0 D_3$$

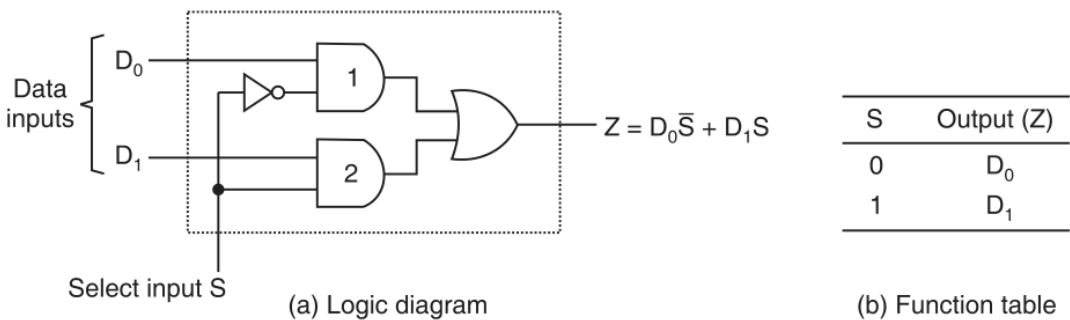


Figure 7.76 2-input multiplexer.

7.24.2 The 4-Input Multiplexer

Figure 7.77a shows the logic circuitry for a 4-input multiplexer with data inputs D₀, D₁, D₂, and D₃, and data select inputs S₀ and S₁. The logic levels applied to the S₀ and S₁ inputs determine which AND gate is enabled, so that its data input passes through the OR gate to the output. The function table in Figure 7.77b gives the output for the input select codes as

$$Z = \bar{S}_1 \bar{S}_0 D_0 + \bar{S}_1 S_0 D_1 + S_1 \bar{S}_0 D_2 + S_1 S_0 D_3$$

Figure

7.77: Logic circuit diagram and truth table of a 4-input multiplexer. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 422 (printed p. 390).

2. Implementing Arbitrary Logic Functions Using Multiplexers

Source: Kumar (Ch. 7, pp. 423–428); Mano (Ch. 4, pp. 201–204)

An **8:1** MUX can implement **any 3-variable Boolean function directly with ZERO external gates**, and can implement **any 4-variable function using only a single inverter!** - *Method*: Connect **(n-1)** variables to the select lines. Group minterms in pairs to determine whether each data input line (**D_i**) should be tied to **0**, **1**, the remaining variable **X**, or its complement **X**.

3. Demultiplexers (DEMUX / Data Distributors)

Source: Kumar (Ch. 7, pp. 430–432, Figure 7.89)

- **Definition**: The exact reverse of a MUX. A DEMUX takes a **single input data line (D)** and directs it to any one of **2ⁿ** output lines depending on the values of **n** select lines. - **The 1-to-4 Line Demultiplexer**:

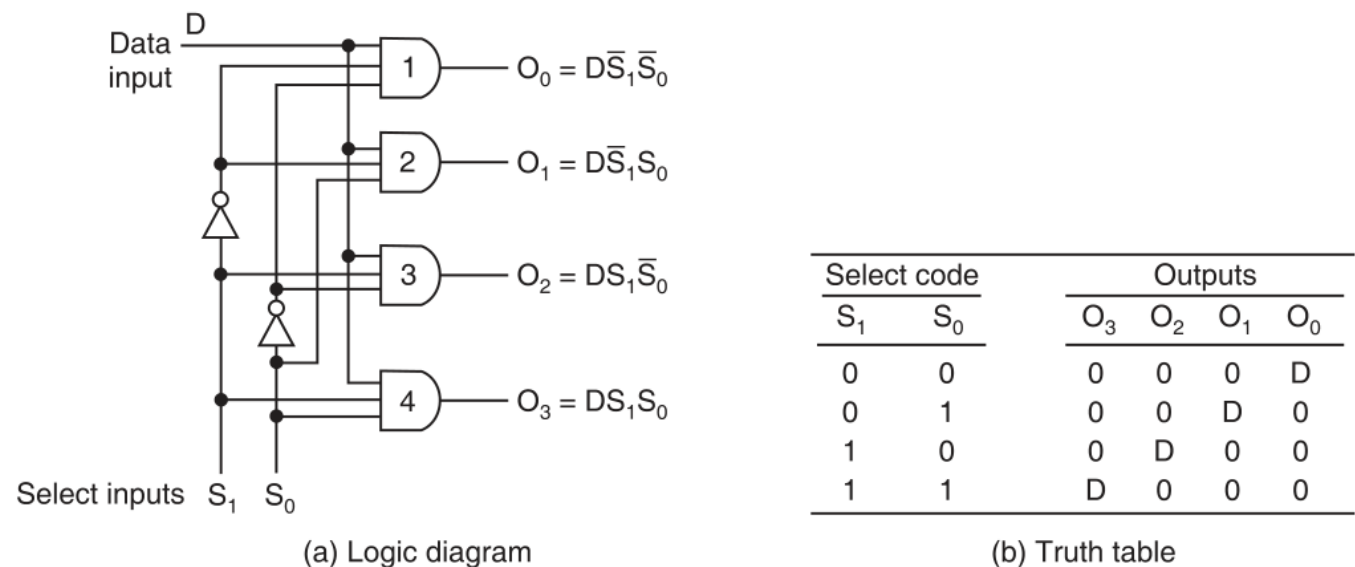


Figure 7.89 1-line to 4-line demultiplexer.

Figure

7.89: Logic diagram of a 1-line to 4-line demultiplexer. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 431 (printed p. 399).

- **Duality with Decoders**: A demultiplexer with data input **D** is **identical in circuitry to a decoder with an Enable input!** If you treat the data line as the Enable input and the select lines as decoder address inputs, the circuits are 100% interchangeable.

PART 3: Decoders, Encoders, Priority Encoders, and Display Drivers

3.8 Decoders and the 74138 IC

Source: Kumar (Ch. 7, pp. 414–418); Mano (Ch. 4, pp. 192–196); Floyd (Ch. 5, pp. 256–264)

[Pacing: Essential Peripheral Building Block — Learn Active-LOW Logic]

Need (Problem-First)

A computer's central processor outputs a binary address code (such as **010**). How does the motherboard know which of the eight connected memory chips is being spoken to? We need a circuit that takes an **n-bit** binary code and activates **one unique output wire** among **2ⁿ** possible output lines. That device is a **Decoder**.

The 3-Line to 8-Line Decoder (Kumar, p. 415, Figure 7.69)

Three inputs (**A**, **B**, **C**) generate **2³ = 8** distinct output minterms (**Y₀** through **Y₇**):

7.69 3-Line to 8-Line Decoder

Figure 7.69a shows the circuitry for a decoder with three inputs and eight outputs. It uses all AND gates, and therefore, the outputs are active-HIGH. For active-LOW outputs, NAND gates are used. The truth table of the decoder is shown in Figure 7.69b. This decoder can be referred to in several ways. It can be called a 3-line to 8-line decoder because it has three input lines and eight output lines. It is also called a binary-to-octal decoder because it takes a 3-bit binary input code and activates one of the eight (octal) outputs corresponding to that code. It is also referred to as a *1-of-8 decoder* because only one of the eight outputs is activated at one time.

Figure

7.69: Schematic diagram and truth table of a 3-line to 8-line decoder with active-HIGH outputs. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 415 (printed p. 383).

- **Active-LOW Outputs in Industry (The 74138 IC):**

In commercial ICs like the **74LS138**, standard NAND gates are used instead of AND gates. Therefore, the selected output line goes **LOW (0 V)**, while all unselected output lines remain **HIGH (+5 V)**. This active-LOW standard exists because bipolar transistors sink current far better than they source it.

3.9 Encoders and Priority Encoders (IC 74148)

Source: Kumar (Ch. 7, pp. 407–413); Mano (Ch. 4, pp. 196–199); Floyd (Ch. 5, pp. 264–268)

[Pacing: Conceptually Heavy — Priority Resolution in Computer Interrupts]

Need (Problem-First)

In a computer system, eight different peripheral devices (keyboard, hard disk, mouse, network card, power monitor) can request service by pulling an interrupt wire. An encoder compresses these 8 wires into a 3-bit binary code telling the CPU which device is calling.

The Critical Failure of a Simple Encoder: What happens if the user presses two keyboard keys at the exact same instant, or the power monitor screams an emergency alert while the mouse is moving? A naive encoder will output garbage (e.g., if line 2 and line 4 are asserted, the encoder might output 6). We need a circuit that **decides which input has the highest priority and completely ignores all lower-priority inputs**. This is a **Priority Encoder**.

The 4-Bit Priority Encoder (Kumar, p. 411, Figure 7.65)

Four inputs (**D₀**, **D₁**, **D₂**, **D₃**) where **D₃** has the highest priority and **D₀** has the lowest priority: - If **D₃ = 1**: Outputs **A₁ A₀ = 11**, regardless of whether **D₂**, **D₁**, **D₀** are 0 or 1 (don't cares)! - If **D₃ = 0** and **D₂ = 1**: Outputs **A₁ A₀ = 10**, regardless of **D₁**, **D₀**. - If **D₃ = 0**, **D₂ = 0**, **D₁ = 1**: Outputs **A₁ A₀ = 01**. - If only **D₀ = 1**: Outputs **A₁ A₀ = 00**. - **The Valid Bit (V)**: What if *no inputs at all* are active? Without an extra bit, the circuit would output 00, falsely claiming that **D₀** was pressed! The **Valid bit (V)** is an extra output pin that equals **1** if at least one input is active, and **0** if all inputs are silent.

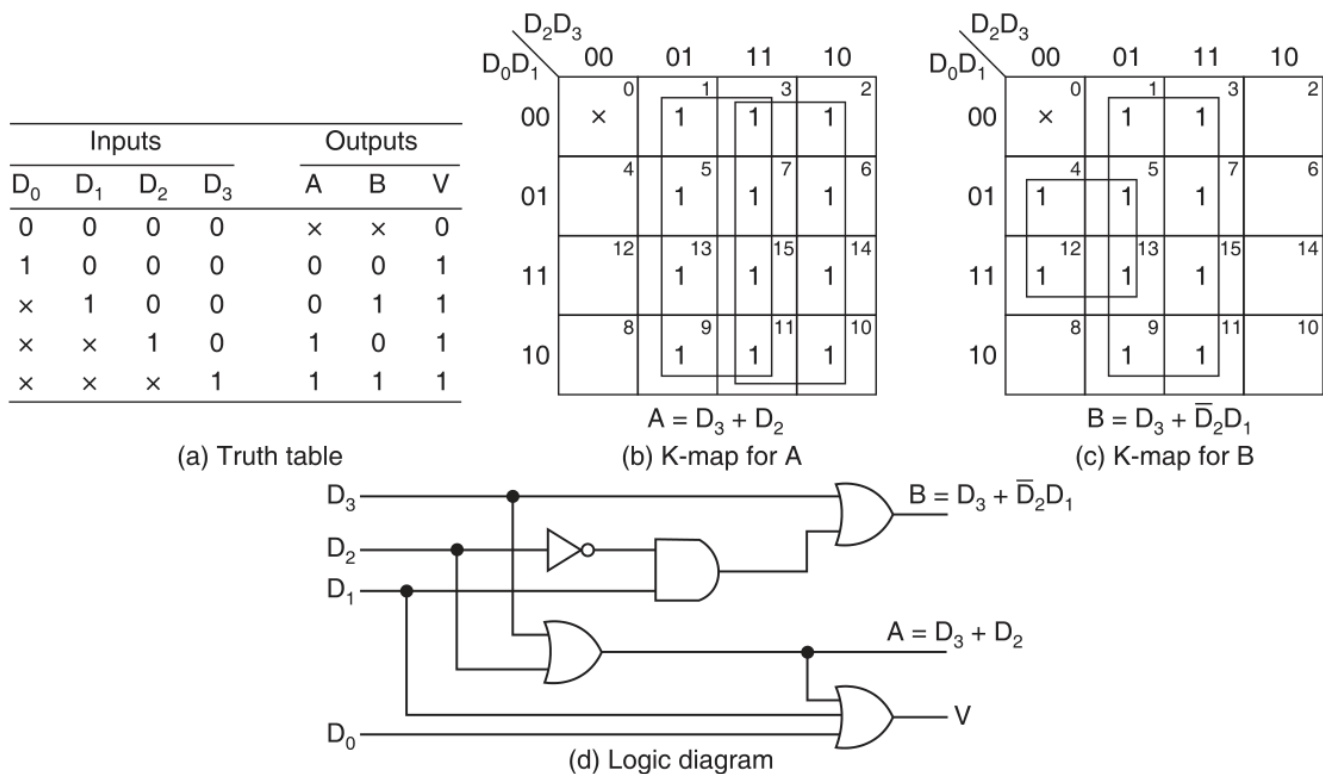


Figure 7.65 4-bit priority encoder.

Figure

7.65: Logic circuitry, truth table with don't care entries, and K-map derivations of a 4-bit Priority Encoder. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 411 (printed p. 379).

- **Industry Standard IC: 74148** (8-to-3 line priority encoder with active-low inputs and outputs, used extensively in microprocessor interrupt controllers).

3.10 Display Drivers: BCD-to-Seven-Segment Decoder/Driver

Source: Kumar (Ch. 7, pp. 419–421, Figures 7.73 & 7.74; Ch. 19, pp. 1013–1031); Floyd (Ch. 5, pp. 260–264)

[Pacing: Visual Hardware Interfacing — High Practical Value]

Need (Problem-First)

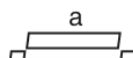
A calculator or digital clock processes numbers as 4-bit BCD codes (0000 to 1001). But a human being cannot look at a panel of four blinking raw binary LEDs and instantly read the time. We need an electronic display device that forms human numbers **0** through **9**, and a specialized digital decoder to illuminate the correct segments.

1. The Seven-Segment Display (Kumar, p. 419, Figure 7.73)

Seven light-emitting diode (LED) bars arranged in a "figure-8" pattern, labeled clockwise from top as **a**, **b**, **c**, **d**, **e**, **f**, with the center bar labeled **g**:

generally provide greater illumination levels but require more power than that by LCDs.

Figure 7.73a shows a seven-segment display consisting of seven light emitting segments. The segments are designated by letters a–g as shown in the figure. By illuminating various combinations of segments as shown in Figure 7.73b, the numbers 0–9 can be displayed. Figures 7.73c and d show two types of LED display—the common-anode and the common-cathode types.



Figure

7.73: The seven-segment display layout, segment illumination combinations for decimal digits 0–9, and internal wiring of Common-Anode vs. Common-Cathode types. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 419 (printed p. 387).

The Two Physical Wiring Configurations:

1. Common-Cathode (CC) Display (Kumar, Figure 7.73d):

The cathode (negative terminal) of all seven segment LEDs are tied together to **Ground (0 V)**.

To illuminate a segment, its individual anode pin must be driven **HIGH (+5 V)**. (Active-HIGH driver like the **7448**).

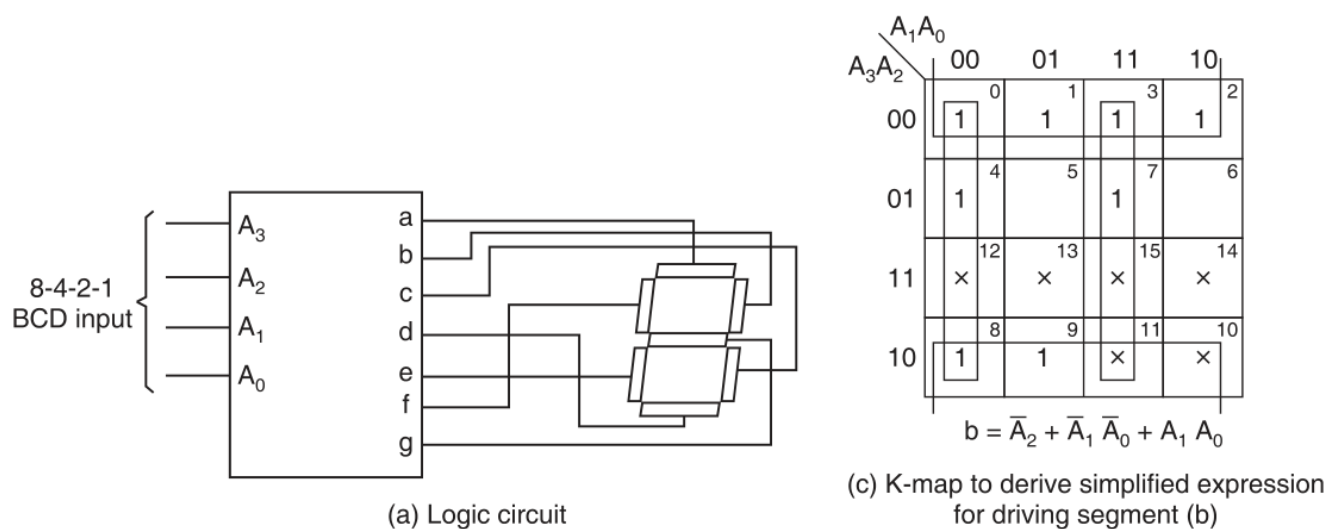
2. Common-Anode (CA) Display (Kumar, Figure 7.73c):

The anode (positive terminal) of all seven segment LEDs are tied together to **+V_{CC} (+5 V)**.

To illuminate a segment, its individual cathode pin must be driven **LOW (0 V)**. (Active-LOW driver like the **7447**). Current flows from **+5 V** through the LED and sinks into the driver pin.

2. The BCD-to-Seven-Segment Decoder/Driver (Kumar, p. 420, Figure 7.74)

- **Inputs:** 4-bit BCD digit **A, B, C, D** (representing numbers **0** to **9**).
- **Outputs:** Seven driving signals **a, b, c, d, e, f, g**.
- **Don't Cares:** BCD inputs **10** to **15** (1010 to 1111) are treated as **Don't Cares (X)** in the K-maps, drastically reducing the gate count!



Decimal digit	8-4-2-1 BCD				Seven segment code						
	A ₃	A ₂	A ₁	A ₀	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

(b) Function table

Figure 7.74 BCD-to-seven segment decoder.

Figure

7.74: Functional block diagram, truth table, and Karnaugh map minimization for segment driver logic. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 420 (printed p. 388).

Example: Segment a Logic Walkthrough

Segment **a** (the top horizontal bar) must be illuminated for digits **0, 2, 3, 5, 6, 7, 8, 9** (it remains OFF only for digits 1 and 4). By plotting these 8 minterms plus 6 don't cares on a 4-variable K-map, the minimized equation is directly derived (Kumar, p. 420).

Unit III Understanding Checkpoints & Misconception Audit

1. Does This Make Sense? Self-Check

- **Question 1:** What is the fundamental physical difference between a Half Adder and a Full Adder?
- **Answer:** A Half Adder can only add two single bits (**A**, **B**). A Full Adder includes a third input for an incoming carry (**C_{in}**) from a lower-order stage, permitting multi-bit parallel arithmetic.
- **Question 2:** Why is a Carry Lookahead Adder (CLA) vastly faster than a standard Ripple Carry Adder?
- **Answer:** In a ripple adder, carries must travel serially through every stage's gates (**2n** gate delays). A CLA calculates all carries simultaneously in parallel using two-level AND-OR logic directly from the primary inputs, executing in just **2** gate delays regardless of word length.
- **Question 3:** If you connect a Common-Anode seven-segment display to a decoder that outputs active-HIGH signals, what will happen?
- **Answer:** The display will invert every number (the segments that are supposed to be ON will stay dark, and the dark segments will light up), because Common-Anode LEDs require an active-LOW (**0 ext{ V}**) sink to turn ON!

2. Conceptual Misconceptions Corrected

- **Misconception:** "A multiplexer can only route data; it cannot do math."
Correction: A multiplexer is a universal Boolean function generator. By connecting input variables to select lines and tying data lines to **0**, **1**, or remaining literals, an **8:1** MUX can implement any 3-variable or 4-variable combinational logic circuit without a single additional logic gate!
 - **Misconception:** "In an adder/subtractor circuit, subtraction requires a completely separate subtractor unit inside the chip."
Correction: There is zero subtraction circuitry. The circuit performs addition on the 2's complement of **B** by using XOR gates to invert **B** and setting the input carry **C₀** = **1**. The adder does not even know it is subtracting!
-